SCORE: Synthesizing Complete, Owned, Responsive Environments for Simulation

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 27 October 2026

Abstract

Games, film, robotics and embodied AI all need 3D environments that can be simulated: every object its own body with collision, mass and joints, under the gravity of the ground it stands on, editable afterwards and owned by whoever made it. World models give frames no simulator opens, and one-shot 3D generators give one fused scene without separate bodies or clear rights. SCORE is a framework that synthesizes such environments. It builds them the way studios build scenes in OpenUSD: a scene sits within its parent scene, and each kind of work on it (layout, surfaces, physics, light, motion, effects, sound, people) is a layer of its own that is checked and changed apart from the rest. A creator locks the design from the top down, an environment concept with its style and its physics and then a concept for each scene; open vision foundation models, procedural tools and coding agents make every layer below, each checked by measured gates before the next is made, and every model allows commercial use of its output, so the creator stays the author. Every step is keyed on its inputs, so an edit runs again only the steps behind it. The result is one OpenUSD environment whose physics a simulator reads as written; Newton, with MuJoCo's solver, steps it. We describe the framework and its method, with a first environment as the evidence; its four demonstrations are marked gaps until its batch has run. Each character also gets a sim body on Newton, driven by its SOMA segment rotations. We close with the route toward bodies true to the person and trained models that drive them. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT license).

Gap: Figure 1, one environment, every layer. Environment 1 from the top down: its physics (the bodies it stands on and their gravity), a top scene on its site, a parent scene with its child scenes, and one scene opened into its layers side by side (base, settle, physics, light, motion, effects, sound, people). It waits on environment 1's batch.

1 Introduction

Simulated 3D environments are raw material for games, film, robot learning and embodied AI [1]. What these uses share is not a look but a structure: separate objects with collision, mass and joints, ground with a gravity, light and sound that come from somewhere, and a scene that can be opened, changed and simulated again. Building one by hand is expensive, and a generator that takes the work over has to leave the result in its maker's hands: in GDC's 2026 survey 52 percent of game workers thought generative AI was hurting the industry [2].

World models generate an environment as frames [3], [4] that no simulator can step. 3D environment generators [5], [6] produce geometry, mostly as one fused asset from which separate objects are recovered after the fact [7], [8]. Both work in one shot, so a fault cannot be traced to the step behind it. Procedural systems [9], [10] and coding agents that write scenes as programs [11], [12],

[13], [14], [15], [16] keep the environment explicit, but rebuild a scene that already exists or draw from a fixed library; their models often float or come apart [17], such agents judge their own geometry unreliably [18], and on scenes rebuilt from video the best model scores 0.91 on static scenes but 0.67 on dynamic ones [19]. Open vision foundation models [20] now read shapes [21], masks [22] and depth [23] out of pictures, and a coding agent can connect them as tools; none gives one style across objects that different models made [24], [25], and none gives physics.

The pieces for the physics exist. OpenUSD [26], the scene description Pixar made for its films, layers each department's work in a file of its own and assembles scenes from smaller ones by reference, and its physics schema states bodies, colliders, mass and joints in the same stage. MuJoCo [27] is a widely used solver for articulated bodies with contact; Newton [28], built on Warp [29], runs MuJoCo's solver on the GPU [30] and reads OpenUSD directly. What is missing is a way to get from what a creator wants to a complete environment in that form.

SCORE is that framework. Its environments are **complete**: every object, surface, light and sound a scene shows exists wherever the viewer looks. They are **owned**: every model in the framework allows commercial use of its output, so the creator holds the full rights. They are **responsive**: every object has its collider, mass and material, and what moves has its joints, so a simulator steps the environment as written. And they are **editable**: each scene is a stack of layers, each layer made by a keyed step, so a change runs again only what it touches. An environment can be a house on a plot of ground or a system of worlds that move one another.

We contribute **the SCORE framework** (Section 2), which turns a creator's locked design into one OpenUSD environment that is physically simulable and opens unchanged in a game engine, a robot simulator or a film renderer; **its method** (Section 3), the stages that build a scene top down and layer by layer, each behind measured gates; and **the evidence plan** (Section 4), four demonstrations on a first environment, left as gaps until its batch has run. Section 5 sets out where the framework goes next, toward simulated people true to the person.

2 The framework

An environment is one OpenUSD record. Each **scene** is one stage; a scene sits within its parent scene and holds its child scenes, referenced at the rooms its plan gives them, so opening a top scene opens everything below it. Each scene's layers stack in a fixed order: a slot for the creator's edit layer on top, then people, sound, light, motion, effects, crops, machines, physics, settle, ground and the base laid from the locked plan. Each object, one per real-world object, carries its kind, transform, collider and each part's library surface. The environment's **physics** is decided in its environment concept (EC): a plot of ground with a gravity and a sun path, one world, or a system of bodies that move one another, a real body taking its values from a cited table and an invented one given by its radius, mass, orbit and spin. The physics layer writes a scene's bodies, mass, joints and the gravity of the body it stands on in OpenUSD's physics schema, so any reader of that schema gets the same scene.

The creator works in the creator cockpit, one signed-in web page, where he locks the design, buys paid work and reviews what the framework reports. Below him, an environment is a table of stages read off its designs; environment 1's has 261 rows. A **node** is one stage for one scene (`assembly@hangar`) or for the environment, and one runner makes, judges and accepts every node. From an approved design everything runs without a further yes.

Keys. A node is made only when its key has no finish record. The key hashes the stage and scene,

its code version, contracts, parameters and seed, the pinned model images, every file it reads and the accepted output of every node it reads. The code version counts only code that can shape the output; code that decides where and how a job runs changes what a run costs, never what it makes, and is in no key. An edit or a fixed fault moves only its own key and those after it, so only those steps run again. Outputs are kept by content hash, never overwritten.

Gates and accepts. Each gate answers pass, fail or unknown, with measures and a reason; missing evidence is unknown, and unknown blocks like fail. The runner accepts an output by itself when every gate passed and it is no worse than the one accepted before. Later nodes read only accepted outputs. The creator keeps four actions: reject an accepted output (every node that read it is then stale), accept a worse one with a reason, waive one gate for one key, and override the runner for one paid run. A store that refuses these records from an agent's process, and a hook, keep agents from them.

Batches. Paid work is bought as a batch with one estimate; the creator's yes sets the batch's budget across all its tries, and the money guards stop it there. Inside a batch every paid job runs once; a failed output becomes a batch gap that the completion gate lists. Every paid stage is priced from a unit cost measured on the ledgers; a stage with no measured run has no price, and its estimate is shown apart as a guess.

3 Method: building an environment

3.1 From concept to plan

The environment concept. The creator first locks the EC in a short interview with Gemini 3.1 Pro [31]: its style and its physics, with his reference pictures, and the scenes it holds. An environment plan places each top scene's site on that ground and works out the sky each site sees.

A concept for each scene, top down. For each scene a concept is then locked, in order of parent scene and child scene. Each draw is one Nano Banana Pro [32] call, on a concept review page where he marks areas and asks for a new draw. A child scene is drawn only once its parent is locked and planned, on a crop of the parent's concept, so it fits the parent it stands in.

Inventory and plan. Gemini 3.1 Pro lists every object of the locked concept as a row of the scene inventory, with its kind, count, anchor and size in metres, and SAM 3 [22] boxes each row; nothing is made or placed that is not a row. Nano Banana Pro then draws the scene from straight above, and Gemini 3.1 Pro writes the plan in metres from that drawing, the concept and the inventory: outline, walls, openings, levels, stairs, roofs, terrain, each item with its facing, people as the spots they stand on, and the child scenes' rooms. Free checks follow (every row placed as often as it counts, inside the outline, no two floor items in each other, every door and standing spot reached on foot, the child scenes tiling their parent); what remains goes back as notes at most twice, and only what is left is a gap. The creator's edits on the plan review page always win and redo what follows.

Shapes first. The route is one loop at every scale: a parent scene gets its concept, plan and a rough model; its plan is split into the shapes of its child scenes, and once those shapes are fixed every child scene is made in parallel, each repeating the loop. The rows `rough-model` and `shell-split` are built and have not run.

3.2 Layers from row kinds

A scene's layers are read off its design alone: each inventory row's kind, found from the reading's noun by one table of rules, says what its objects do (emit, move, lie loose, glow, grow, act, sound). A scene gets an effects, motion, physics, crops or people layer only for rows that do so, and light and sound always; each layer is made from the rows by their kind's rule, so a new environment needs no hand-written list. The scene record holds what no made piece covers (shells, stairs, ground and water, backdrop, lamps, neighbours), gated on the shell lying within 5 cm of the plan and every doorway opening onto a floor.

Models. Every object goes the prop route first: Nano Banana Pro draws its close-up from its crop of the concept (redrawn at most three times when the shape check rejects it), Pixal3D [21] builds it, and a likeness gate compares its outline from eight sides with the close-up. Only an object whose checks fail is built by a code builder a coding agent writes; a thin straight thing (a pipe, post or beam) whose every close-up failed is swept along its length in code.

Surfaces. Everything a scene shows comes from the creator's library, score_lib, or is a written exception. A model gives shape only: each part takes one surface from a library of rule-based surfaces, written as ProcFunc functions [33] on Infinigen's shaders [9] and coloured from the scene's palette. The finish route labels each face from free renders of the model, matches each finish to a library surface, bakes it and checks the bake against the close-up (each finish within ΔE 32 of its colour there). A free library check traces every material, model, light, sound and character of an accepted assembly to the library or an exception.

Settle and physics. Loose objects are dropped onto the made ground; one that would turn more than 15 degrees or drift more than 30 cm is a resting fault and keeps its laid pose. The settle lays where each object comes to rest as a layer. The physics trial then steps the composed scene on Newton with MuJoCo's solver on Newton's contacts, under the gravity of the body the scene stands on, and judges it by gates: `reacted`, `at rest`, `energy` (no rise in undriven energy) and `hinge` (a hinged leaf swings to its limit). The trial moved from Blender's Bullet [34] to Newton on 27 October 2026. On a toy scene at Moon gravity (two crate drops, a driven door with a wheel fixed to it, a pushed stool), both engines gave the same answers on every gate, and Newton landed the crates on their targets where Bullet left its contact margins of 8 mm; with Newton 1.7.0 in its deterministic mode, six runs gave bitwise equal bodies in every frame. The settle moves to Newton the same way, so one engine steps what comes to rest and what is tried, and one effect is stepped on Newton as a first case for the effects layer.

Light, motion, effects and sound. The sky is worked out from the EC's physics, never drawn: each body at its true direction, size and phase as seen from the scene's site. The lamps are the inventory's glowing rows; one render per scene gives each lamp its own light group, and a free sweep mixes their strengths to bring the scene within 1.5 times its concept's brightness. Every object of a moving row moves by its kind's rule at a speed cited from real machines, gated on nothing passing within 3 cm of another thing; emitters are measured against real footage; each sound source gets a library file or one MOSS-SoundEffect [35] take, gated on a CLAP [36] score of at least 0.30.

People. People are models like any other: a close-up per person row, built by Pixal3D and painted by the finish route. SOMA's body [37], read from the close-up by SAM 3D Body [38], is fitted under it as its rig, and Kimodo's clips [39] move it; a shell gate passes a look when 99 percent of its soft points lie within 5 cm of the body. Each person stands on a person spot of its scene's plan.

Sim bodies. Each character also gets a **sim body**, its physical twin on Newton, so people in an

environment are simulated, not only animated. The minimal body is built straight from SOMA's 23 segments with a ball joint at each SOMA joint frame, driven by the character's SOMA segment rotations with no scaling and no per-person setup, its mass spread by its segment volumes, standing on a SCORE floor so its ground forces come from contact; muscles enter as a minimal first step. Built this way, the body follows SOMA's rotations to within 6.9e-9 m, and on Newton it matches plain MuJoCo to 3.4e-8 after 200 steps.

Gap: Newton for the settle, one effect and the sim body. The settle and the physics trial on one engine, with the resting and physics gates per scene against the Bullet settle; the one effect stepped on Newton; and a sim body walking on a SCORE floor, its joint angles and ground forces against OpenCap's LabValidation motion capture and force plates [40]. It waits on these three stages being built and run on a cloud card.

Completion. A scene is complete when its gate passes on its final assembly: every row made or dropped with a reason, nothing unmade, no resting fault, every layer and shown scene present. A fault the creator ships with is an accepted gap, written with who decided, when and why; the gate lists every gap by name and is never loosened. A hook stops an agent claiming a scene is done while its gate fails, since agents judging their own scenes agree with a measured geometry score only 45.8 percent of the time [18]. A scene's films are made from scripts the creator locked, once it is complete.

4 Evidence: environment 1

Environment 1 is the evidence. Its first run was not accepted by the creator and its store was lost on 19 October 2026, so this draft shows none of its outputs; each demonstration below is made on its new batch. Figure 1 is the first.

Gap: Demonstration 2, physics that holds. Environment 1's scenes stepped on Newton under their own body's gravity: loose objects come to rest, doors swing on their hinges, with the physics trial's gates per scene and Bullet run on the same scenes as the baseline. It waits on environment 1's batch and on Newton taking over the settle.

Gap: Demonstration 3, one environment, many uses. The same OpenUSD environment opened unchanged in a game engine, in a robot simulator and in a film render, with what each reads of its layers and what it drops. It waits on environment 1's batch and on the engine adapter.

Gap: Demonstration 4, an edit that stays cheap. The creator changes one thing in a locked scene, and only the nodes whose keys it moves run again; the nodes rerun, time and euros against a full rebuild. It waits on environment 1's batch and on the edit layer being carried from one assembly to the next.

Gap: Table 1. Per scene of environment 1: inventory rows, models generated and code-built, layers, gaps before and after the batch, time, euros and the creator's verdict. It waits on environment 1's batch and the creator's review.

Disclosure. The creator's locks, picks, accepts and waivers are his own clicks; everything between them runs automatically. Coding agents wrote most of the framework's code under the author's direction: 2,413 of its 2,436 commits carry a coding agent as co-author. Hand fixes to a scene are recorded as the creator's edits.

5 Future work: people true to the person

The minimal sim body is a start. This section states goals, not results.

Bodies true to the person. The sim body is to become fully musculoskeletal and true to the actual person, older people and a curved spine included, not a template body resized: joint centres from bone landmarks on its SOMA mesh, then mass, muscles, joint ranges, spine and bone shape, each input SOMA lacks marked as assumed with a range. A full musculoskeletal body with 416 muscles already runs on Newton but still drifts 1.5 cm from plain MuJoCo after 300 steps, so Newton is to be validated for bodies before more is built on it. Its joint angles, then its joint moments and ground forces, are to be held to OpenCap's LabValidation data.

Scenes from motion. Motion is to come from the SOMA ecosystem: GEM-X [41] from video, Kimodo from sentences, and sequences joined from datasets of both. The terrain a motion implies becomes a SCORE scene and a sim body follows the motion on it, so the scene keeps the motion with no video and nothing that identifies a person.

Brains. A **brain** is a trained model that drives a sim body's muscles so it moves on its own, including over structures it never saw. Trained on sim-body data and placed in SCORE's invented worlds, under their own gravity and ground, it is to adapt to what no recording holds. Specialised models, for example for clinical groups, would be trained outside SCORE on data that never enters it.

The data loop. Sim bodies moving in synthesized scenes make data; that data trains better and more specialised brains; a brain drives a sim body in new scenes, which makes more data.

6 Limitations and conclusion

One creator judges, and he has not yet accepted a run. Closed models (Nano Banana Pro, Gemini 3.1 Pro) draw the concepts and close-ups and read and plan every scene; they may change or go. The design step, the layers from row kinds, the finish route, the shapes-first loop and the scene tree have not yet carried a scene through a batch; the edit layer is a slot, the settle, the effects and the sim body are not yet on Newton, and no game engine or robot simulator has opened the result yet. Next: environment 1's batch and its four demonstrations, an engine adapter, the edit layer carried across assemblies, more environments, open models in place of the closed ones. In short, SCORE synthesizes an environment the way studios build one, scene by scene and layer by layer, in a form a simulator steps as written; an accepted environment will show whether it holds.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[12] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[13] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[15] G. He, R. Luo, W.-C. Ma, and H. Averbuch-Elor, "Thinking in Blender: Staged executable inverse graphics with vision-language models," *arXiv preprint arXiv:2606.02580*, 2026.

[16] H. Wang *et al.*, "Code as worlds: Agentic discovery of executable world representations for physical reasoning," *arXiv preprint arXiv:2608.27549*, 2026.

[17] Y. Gao *et al.*, "3DCodeBench: Benchmarking agentic procedural 3D modeling via code," *arXiv preprint arXiv:2606.01057*, 2026.

[18] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[19] R. Shen *et al.*, "4DCodeBench: Benchmarking agents on inverse graphics of dynamic scenes," *arXiv preprint arXiv:2610.03715*, 2026.

[20] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[21] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[22] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[23] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[24] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[25] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[26] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[27] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ international conference on intelligent robots and systems (IROS)*, 2012, pp. 5026–5033.

[28] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[29] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[30] Google DeepMind and NVIDIA, "MuJoCo Warp: A GPU-optimized version of the MuJoCo physics simulator." 2026. Available: https://github.com/google-deepmind/mujoco_warp

[31] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[32] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[33] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[34] E. Coumans and Y. Bai, "Bullet physics SDK and PyBullet." 2021. Available: https://github.com/bulletphysics/bullet3

[35] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[36] Y. Wu, K. Chen, T. Zhang, *et al.*, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," *arXiv preprint arXiv:2211.06687*, 2022.

[37] J. Saito, J. Li, M. de Ruyter, M. Guerrero, *et al.*, "SOMA: Unifying parametric human body models," *arXiv preprint arXiv:2603.16858*, 2026.

[38] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[39] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[40] S. D. Uhlrich *et al.*, "OpenCap: Human movement dynamics from smartphone videos," *PLOS Computational Biology*, vol. 19, no. 10, p. e1011462, 2023.

[41] NVIDIA, "GEM-X: Monocular whole-body 3D human pose estimation using the SOMA body model." 2026. Available: https://github.com/NVlabs/GEM-X